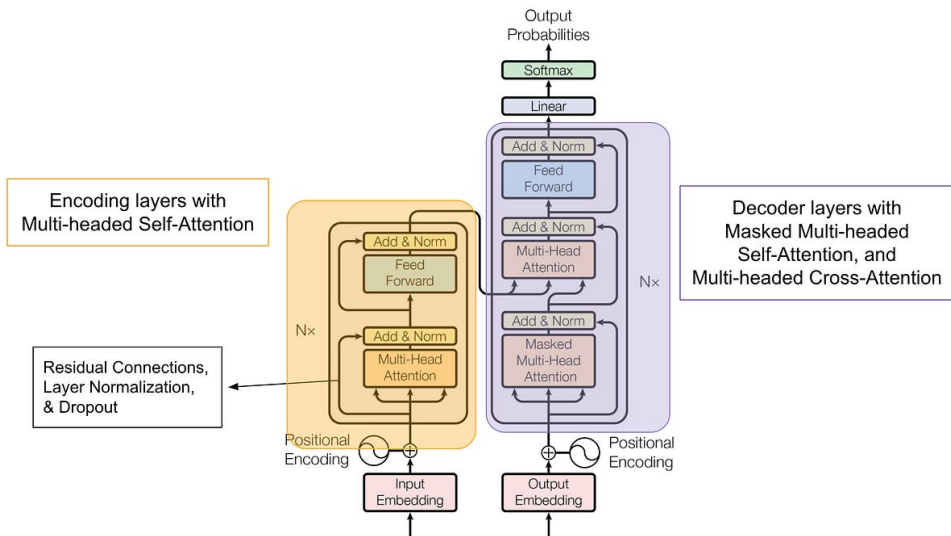




How Transformers Actually Work

From neural networks to RNNs to LSTMs to attention: the full evolution, and why each one broke.

NN → RNN → LSTM → Seq2Seq → Attention →  Transformer



Roadmap: 6 Architecture Steps

Each architecture solved the last one's fatal flaw

1. Feedforward Neural Net

Can't handle variable sequences at all

1980s+

2. Recurrent Neural Net (RNN)

Handles sequences, but suffers vanishing memory

1990s

3. LSTM / GRU

Remembers longer, but strictly sequential (slow)

1997

4. Seq2Seq Encoder-Decoder

Translation works, but fixed vector bottleneck

2014

5. Attention Mechanism

Fixes bottleneck via dynamic lookup, still slow

2014-15

6. The Transformer

Drops recurrence completely: fully parallel

2017



The Through-Line: Every step exists because the previous architecture hit a concrete wall. Understanding *what broke* makes the Transformer's design feel inevitable.



AI Coach John

Follow



 Repost

Step 1: Feedforward NNs

Where it starts: and why it can't read a sentence

A basic neural network takes a **fixed-size** input, passes it through layers of weighted connections and activation functions, and produces a **fixed-size** output.

FEEDFORWARD PIPELINE

```
[input: fixed size] -weights-> [hidden: learned] -weights->
[output: fixed size]
```

Variable Input Size Mismatch

"The cat sat" (3 words) → Fits slot

"The cat sat on the mat quietly" (7 words) → Fails: Overflow!

✗ Why it fails on language:

- ✗ **Variable length:** "Hi" vs 500-word essay can't fit same slot.
- ✗ **No order sense:** "dog bites man" ≡ "man bites dog".
- ✗ **No memory:** Each prediction is completely independent.



Step 2: RNNs: Adding Memory

Process one word at a time, carry state forward

An RNN processes a sequence step by step, maintaining a **hidden state** that carries information from previous steps into the current one.

Unrolled RNN Sequence Chain

"The" → [RNN] – **h1** → "cat" → [RNN] – **h2** → "sat" → [RNN] → Output
Hidden state passed forward each step

✓ What it solved:

- ✓ **Handles variable-length sequences naturally:** Just keep stepping.
- ✓ **Order matters:** State accumulates sequentially.
- ✓ **Shared weights at every timestep:** Efficient & generalizes well.



⚠ STEP 2 FLAW

Why RNNs Break

The flaw that made long sequences impossible

During training, gradients flow backward through every timestep. Each step multiplies them by a value: and across many steps, those multiplications compound.

gradient at step 50 $\leftarrow \times \leftarrow \times \dots \leftarrow$ Loss (50 multiplications)

If factor < 1 : $0.9^{50} = 0.005 \rightarrow$ **VANISHES** (early words learn nothing)

If factor > 1 : $1.1^{50} = 117.0 \rightarrow$ **EXPLODES** (training destabilizes)

Gradient Signal Decays Exponentially

step 1

step 10

step 25

Practical Consequence: "The cat, which had been sleeping on the warm mat all afternoon, was ____"

RNN has effectively forgotten "cat" by the time it gets here.

The model can't connect information across long gaps: exactly what language requires.



AI Coach John

Follow



Repost

Step 3: LSTM Memory

Long Short-Term Memory: gates that decide what to keep

LSTM (1997) adds a **cell state**: a separate memory channel that flows through the sequence with minimal modification: plus three **gates** that learn to control it.

Forget Gate

What to discard from memory

Input Gate

What new info to store

Output Gate

What part to expose

Cell State: Memory Highway across timesteps

Why this fixes vanishing gradients: The cell state passes forward with mostly *addition* rather than repeated multiplication: so gradients can flow across many timesteps without exponentially decaying.



GRU (2014): A simplified variant with two gates (Reset & Update) instead of three, performing comparably with less compute.



AI Coach John

Follow



 Repost

⚠ STEP 3 BOTTLENECK

Why LSTMs Weren't Enough

Better memory, same fundamental bottleneck

LSTMs genuinely improved long-range memory: but two problems remained:

1. Sequential Processing (The Killer Limit)

word1 → word2 → word3 → ... → word1000

each step MUST wait for the previous one

Can't parallelize across the sequence → can't use GPUs efficiently → can't train on internet-scale data in reasonable time.

⌚ 2. Still Degrades Over Very Long Distances

Gates help, but connecting information across hundreds of tokens remains unreliable: memory is compressed into a fixed-size state that inevitably loses detail.

LSTM: word1 → word2 → word3 ... (sequential, GPU mostly idle) ⌚

Ideal: all positions processed at once ⚡

⚡ This is THE constraint the transformer was built to break.



AI Coach John

Follow

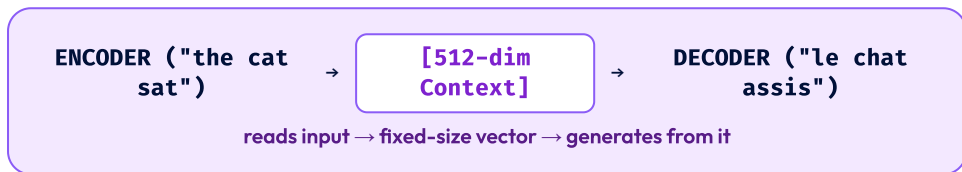


Repost

Step 4: Seq2Seq Models

Two networks: one to read, one to write

For translation, a single sequence model isn't enough: input and output are different lengths in different languages. Seq2Seq (2014) used two RNN/LSTM networks:



The Bottleneck Problem

The encoder must compress the **entire** input sentence into one fixed-size vector, and the decoder sees only that.

A 5-word sentence → 512-dim vector (✓ fine)

A 100-word sentence → 512-dim vector (✗ massive information loss!)

[word][word][word][word][word][word][word] → | 512 dims | → [out][out][out]

Everything squeezed through one narrow channel



Translation quality dropped sharply as sentences got longer: everything was squeezed through one narrow channel.



Step 5: Attention

Let the decoder look back at every input word

Instead of one compressed vector, attention (2014-15) lets the decoder access **all** encoder states, weighting how much each input word matters for the current output word.

Generating "chat" (French for cat):



→ weighted sum = context for THIS output word

★ Why it worked so well:

- ✓ **No fixed-size bottleneck:** Decoder draws directly from full input.
- ✓ **Dynamic context:** A different context for every generated word.
- ✓ **Interpretable:** You can visualize which words the model focused on.



Step 6: The Transformer

Drop recurrence entirely: Attention Is All You Need

The transformer paper's radical move: if attention already connects any position to any other position, **why keep the RNN at all?**

BEFORE (RNN + Attention): sequential recurrence + attention on top

AFTER (Transformer): attention only: no recurrence whatsoever

🔒 What removing recurrence unlocked:

- ⚡ **Full parallelization:** All positions processed simultaneously (GPUs 100%).
- 🔗 **Constant path length:** Pos 1 and Pos 1000 are 1 attention step apart.
- 📏 **Trainable at scale:** Made training on internet-scale data feasible (LLMs).

RNN: Pos 1 to 1000 = 999 steps

TRANSFORMER: Pos 1 to 1000 = 1 step ⚡



Self-Attention: Q, K, V

The core mechanism, explained concretely

Every token produces three vectors via learned weight matrices:



Query (Q): "what am I looking for?"



Key (K): "what do I contain?"



Value (V): "what do I actually contribute?"

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

match queries against keys → scale (stability) → weighted sum of values



The Library Analogy: Your Query is what you search for. Each book's Key is its title. You match your Query against all Keys for relevance, then read the Values (contents) weighted by relevance.



Multi-Head Attention

Running attention several times in parallel

Rather than one attention computation, transformers run several **heads** in parallel: each with its own Q/K/V projections: then concatenate the results.

Parallel Attention Projections & Concatenation

 Head 1: Syntax (Subject-Verb)

 Head 2: Coreference ("it" → "cat")

 Head 3: Positional / Local

 Head N: Global Semantics

concatenate → project → output

Why multiple heads:

A single attention pattern forces the model to average all relationship types into one weighting. Separate heads let the model attend to grammar, meaning, and position *simultaneously* rather than compromising between them.



Positional Encoding

Attention has no sense of order: so we add it explicitly

Self-attention treats its input as a **set**, not a sequence. Without position information:

"the dog bit the man" & "the man bit the dog" → **identical to raw attention (X)**

The fix: add position information directly into each token's vector before attention.

```
token_embedding = [0.21, -0.44, 0.87, ...]
positional_enc  = [0.01, 0.99, 0.03, ...] # position 3
model_input     = token_embedding + positional_enc
```

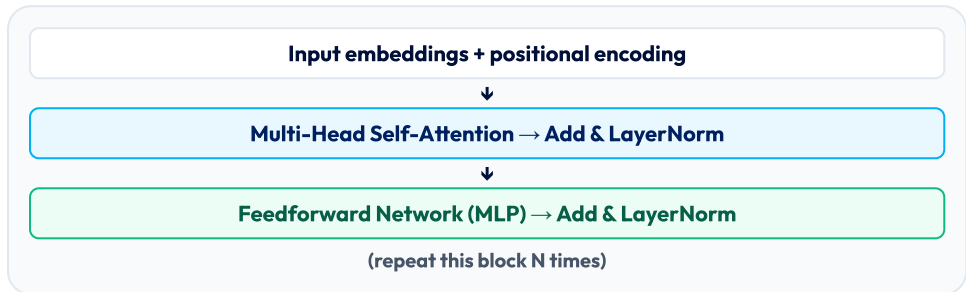


Modern approaches: Original paper used sine/cosine functions; most current LLMs use **RoPE (Rotary Position Embedding)**, which encodes relative position and generalizes far better to long sequences.



The Transformer Block

How the pieces stack together



Why the 2 supporting pieces matter:

- ✓ **Residual connections (the "Add"):** Let gradients flow directly through network, making it possible to stack dozens of layers without training collapsing.
- ✓ **Layer normalization:** Stabilizes scale of activations between layers, keeping training stable at depth.



Three Architecture Families

Same building blocks, three different assemblies

Encoder-Only (BERT)

Bidirectional attention: each token sees full context both ways. Best for *understanding* (classification, embeddings, NER).

Decoder-Only (GPT, Llama, Claude)

Causal/masked attention: each token sees only what came before. Best for *generation* (what every modern LLM uses).

Encoder-Decoder (T5, Original Transformer)

Encoder reads input fully, decoder generates output while attending. Best for *transformation* (translation, summarization).





Why Decoder-Only Won

The architecture behind every model you use today

- ✓ **One simple training objective:** Predict next token. No masking scheme to design, no encoder/decoder split to balance.
- ✓ **Any text becomes training data:** The entire internet is usable as-is, no labeled pairs needed.
- ✓ **Scales cleanly:** The same architecture works at 1B and at 1T parameters.
- ✓ **Generality emerged:** Tasks (translation, summarization, Q&A) became "predict next token given right prompt".

The Causal Mask Concretely

Generating tok 4: [tok1] [tok2] [tok3] [tok4] [MASK] [MASK]
(✓ can't peek at future during training)



⚠ SCALABILITY LIMITS

Quadratic Cost ($O(N^2)$)

The transformer's own fundamental limitation

Every token attends to every other token: so cost grows with the **square** of sequence length.

1,000 tokens → 1,000,000 attention pairs
10,000 tokens → 100,000,000 attention pairs (*100x cost for 10x len!*)
100,000 tokens → 10,000,000,000 attention pairs

✂ How the field responded:

- ✓ **Flash Attention:** Tiled memory compute; identical results, less RAM traffic.
- ✓ **GQA / MQA:** Share key/value heads across query heads to shrink KV cache.
- ✓ **Sparse / Sliding Window:** Token attends to a subset rather than all.
- ✓ **State-Space Models (Mamba):** Linear scaling alternative architecture.



AI Coach John

Follow



Repost

SUMMARY MATRIX

The Evolution Recapped

Every architecture and what it fixed

Architecture	Key idea	Solved	Broke on
Feedforward NN	Fixed layers	Basic learning	Variable length
RNN	Hidden state	Sequences	Vanishing gradient
LSTM / GRU	Gated memory	Long memory	Sequential (slow)
Seq2Seq	Encoder-Decoder	Translation	Fixed bottleneck
Attention	Weighted lookup	Bottleneck	Sequential base
Transformer	Pure attention	Parallel GPU scale	$O(N^2)$ Cost



Save this reference guide. Follow for more AI engineering breakdowns.



AI Coach John

Follow



 Repost